

LAB GUIDELINES

Lab 26 — Backup Strategy: Full, Synthetic, Incremental, Differential, Snapshot

Maps to CompTIA Server+ objective **3.7 — Explain the importance of backups and restores** (backup methods: full, synthetic full, incremental, differential, archive, open file, snapshot; backup frequency; media rotation; backup media types: tape, cloud, disk, print; file-level vs. system-state backup; restore methods: overwrite, side by side, alternate location path; backup validation: media integrity, equipment, regular testing intervals; media inventory before restoration).

Run all commands on <https://killercode.com/playgrounds/scenario/ubuntu>

Required software (free, apt): tar, rsync, restic, lvm2

```
apt update && apt install -y rsync restic lvm2
```

Step 1 — Prepare a "live" data set and a backup target

```
mkdir -p /srv/data /srv/backup
for i in $(seq 1 10); do dd if=/dev/urandom of=/srv/data/file$i.bin bs=1K count=$((RANDOM%50+5)) 2>/dev/null; done
du -sh /srv/data
```

Step 2 — Full backup with `tar`

```
tar -czf /srv/backup/full-$(date +%F).tgz -C /srv data
ls -lh /srv/backup/
```

A **full** backup captures everything every time. Largest size, simplest restore.

Step 3 — Incremental backup with `tar` snapshot files

```
echo "more data" > /srv/data/file11.txt
tar --listed-incremental=/srv/backup/snap.snar -czf /srv/backup/inc-$(date +%F).tgz -C /srv data
ls -lh /srv/backup/
echo "newer data" > /srv/data/file12.txt
tar --listed-incremental=/srv/backup/snap.snar -czf /srv/backup/inc-$(date +%F)-2.tgz -C /srv data
```

Incremental = changes since the **last** backup (full or incremental). Smallest backup, **longest** restore (need full + every incremental in order).

Step 4 — Differential backup

```
cp /srv/backup/snap.snar /srv/backup/snap-full.snar # freeze a full marker
echo "diff change" > /srv/data/diff1.txt
tar --listed-incremental=/srv/backup/snap-full.snar -czf /srv/backup/diff-$(date +%F).tgz -C /srv data
```

Differential = changes since the last **full** backup. Bigger than incremental, faster restore (need only full + latest differential).

Type	Size growth	Restore complexity	Time-to-restore
Full	Largest every time	1 file to read	Fastest
Incremental	Smallest	Full + N incrementals	Slowest
Differential	Grows daily	Full + latest differential	Medium
Synthetic full	Server constructs a new "full" from full+incrementals — clients only ever upload increments	1 file	Fast

Type	Size growth	Restore complexity	Time-to-restore
Snapshot	Copy-on-write delta	Instant	Instant (same-host)

Step 5 — Synthetic full via `restic`

`restic` is content-addressed deduplicated backup — every snapshot acts like a full but only the new chunks are stored.

```
export RESTIC_PASSWORD='LabRestic!1'
restic init --repo /srv/backup/restic
restic -r /srv/backup/restic backup /srv/data
echo "extra" > /srv/data/extra.txt
restic -r /srv/backup/restic backup /srv/data
restic -r /srv/backup/restic snapshots
```

Each `restic` snapshot looks like a full but stores only the delta.

Step 6 — Snapshot (LVM)

Already shown in Lab 3. Snapshots are **same-host** and **not a backup** — if the host dies, the snapshot dies with it.

Step 7 — Open-file backup

Databases that write while you copy them give you a corrupt image. Server+ 3.7 lists **open file** as a backup method — the answer is **VSS on Windows** or **application-consistent dump on Linux**:

```
# MariaDB live consistent dump:
# mysqldump --single-transaction --routines --triggers --all-databases | gzip > /srv/backup/mariadb.gz
echo "mysqldump --single-transaction is the open-file-safe path for InnoDB"
```

For arbitrary filesystems, take an LVM or filesystem snapshot first, then back up the snapshot (consistent point-in-time view of all files).

Step 8 — File-level vs. system-state

Backup type	What it captures	Restore use case
File-level	Individual files / directories	Single-file recovery
System-state	OS, registry/AD database, boot files, IIS config, certstore	Full server rebuild
Bare-metal image	Entire disk image	Disaster recovery to new HW

Step 9 — Backup media types

Medium	Pros	Cons
Tape	Cheapest \$/TB, very long shelf life	Slow restore, robotic library cost
Disk	Fast, random access	Online → ransomware-reachable
Cloud	Off-site, scalable	Egress cost, network dependency
Print	Air-gapped paper record	Tiny capacity — only used for keys / runbooks

Step 10 — Rotation: GFS (Grandfather-Father-Son)

```
Son      = daily incrementals (7 kept)
Father   = weekly fulls (4 kept)
```

Grandfather = monthly fulls (12 kept)

Plus **off-site** rotation: at least one weekly full leaves the building.

Step 11 — Restore methods

```
# Overwrite restore
tar -xzf /srv/backup/full-$(date +%F).tgz -C /srv

# Side-by-side restore
mkdir -p /srv/restore-test
tar -xzf /srv/backup/full-$(date +%F).tgz -C /srv/restore-test

# Alternate-location restore (restic)
restic -r /srv/backup/restic restore latest --target /srv/restore-alt
```

Server+ names exactly these three: **overwrite**, **side-by-side**, **alternate location path**.

Step 12 — Backup validation (the part that gets skipped — and shouldn't)

Server+ 3.7 lists: media integrity, equipment, regular testing intervals, media inventory before restoration.

```
restic -r /srv/backup/restic check                # media integrity
restic -r /srv/backup/restic snapshots            # inventory before restore
restic -r /srv/backup/restic restore latest --target /tmp/verify --dry-run
diff -r /srv/data /tmp/verify/srv/data && echo "restore matches source"
```

Schedule a quarterly restore drill — an untested backup is no backup.

Free online tools

- **restic docs** — <https://restic.readthedocs.io/>
- **BorgBackup** — <https://www.borgbackup.org/>
- **Duplicati** — <https://www.duplicati.com/>
- **Bacula / Bareos** — <https://www.bareos.org/>

What you learned

- Full / incremental / differential / synthetic full / snapshot — what each costs and how restore differs.
- Open-file backup with application-consistent dumps and snapshots.
- File-level vs. system-state vs. bare-metal image backups.
- Backup media trade-offs and GFS rotation.
- The three Server+ restore methods (overwrite, side-by-side, alternate location).
- Backup validation: integrity check, inventory list, regular restore drill.